MIDTERM EXAM

CAP 5610: Introduction of Machine Learning — **Assigned:** 9 October 2026
Dr. Adigun — **Due:** 19 October 2026

Directions: This is a take-home project. Your task is to train different machine learning models and report their performance. You will complete three tasks. Submit a report (no more than 2.0 pages). For each question, describe the best-performing models, their parameters, and their performance. Also highlight any trends you observed. Place all Figures and Tables in the Appendix. There is no page limit for the Appendix. *Optional: You may submit one Jupyter notebook per task and include the best parameters you identified.*

1. **Logistic Regression (30 points):** Your task is to build a binary classification model using the Adult Census Income dataset. This data contains demographic features to predict whether an individual earns more than $50,000 annually. Here are the features:

   - `age` (Continuous)
   - `workclass` (Categorical)
   - `education` (Categorical)
   - `education-num` (Continuous): The numerical equivalent of the education level
   - `marital-status` (Categorical)
   - `relationship` (Categorical)
   - `race` (Categorical)
   - `sex` (Categorical)
   - `capital-gain` (Categorical)
   - `capital-loss` (Categorical)
   - `hours-per-week` (Continuous)
   - `native-country` (Categorical)
   - `label->50K`: This is the target. It is 0 if the income is less than $50,000 and 1 otherwise.

   (a) Preprocess the provided training and test sets, handle missing values, and train a Logistic Regression model. The dataset is imbalanced, so apply the `class_weight='balanced'` parameter during training to force your model to prioritize the minority class.

   (b) After you train your model, extract and interpret the beta coefficients. Identify the features with the largest positive and negative weights. Write a brief explanation detailing what these coefficients mean mathematically.

   (c) Before you evaluate your model's actual performance, establish a "by chance" baseline. Calculate the accuracy you would achieve if you completely ignored the features and simply guessed the majority class for every person. Your model must definitively beat this naive baseline to prove it learned meaningful patterns.

   (d) Finally, generate predictions on the test set and compute three specific metrics: Accuracy, the F_1-Score, and the ROC-AUC. Standard accuracy alone misleads evaluators because

this dataset naturally skews toward the majority class. The F_1-score and AUC will provide a much more rigorous, holistic assessment of how well your model identifies the minority high-income class.

(e) Repeat (a)–(d) but do not use the `class_weight='balanced'` parameter.

2. **Neural Network Regressor (30 points):** Build a neural network regression model using the King County housing dataset to predict continuous house prices. Preprocess the provided data by handling missing values and strictly standardizing your numerical features. Critically, apply a log transformation (like `np.log1p`) to your target variable before training. This step stabilizes the variance and helps the network learn more effectively because real estate prices are heavily right-skewed.

   (a) Design your network architecture and explicitly compare the Logistic sigmoid and ReLU activation functions within your hidden layers. Train a version of your model with each function. Document which activation converges faster and yields better predictions.

   (b) Finally, evaluate your ultimate model's performance on the test set. Do not forget to exponentiate your predictions back to their original monetary scale (using the inverse log, like `np.expm1`) before calculating your error. Compute and report the Mean Squared Error (MSE) and the R^2 score to objectively quantify how accurately your network predicts real-world property values compared to a naive baseline average.

3. **Neural Network Classifier (40 points):** Build a multi-class neural network classifier using the CIFAR-10 dataset. This dataset contains 60,000 color images (32×32 pixels) categorized into 10 mutually exclusive classes (such as airplane, car, bird, cat, deer, dog, frog, horse, ship, and truck). Preprocess the provided training and test sets by flattening the images and strictly normalizing the pixel values between 0 and 1 to ensure stable and efficient model training.

   (a) Design a flexible neural network architecture and systematically vary the number of hidden layers and the number of neurons per layer. Experiment with shallow, wide networks versus deep, narrow networks. Document how increasing the model's overall capacity impacts both training time and the network's ability to extract complex image features. Do not go beyond 8 hidden layers and 1000 neurons per layer.

   (b) Implement a regularization technique because high-capacity networks easily overfit on complex image data. Apply either dropout to randomly deactivate neurons during training, or use ℓ_2 regularization (weight decay) to mathematically penalize excessively large weights. Test and report whether your chosen regularizer successfully reduces the gap between your training loss and validation loss.

   (c) Finally, evaluate your fully optimized model on the test set. Compute and report the overall accuracy and the cross-entropy loss. Additionally, generate a confusion matrix to identify which categories your network struggles to distinguish the most.